

Homework 1: Secure Messaging

*Instructor: Eysa Lee**Due: October 21, 2026*

In this assignment, you will combining cryptographic primitives we've learned about to construct a secure communication framework. This systems prevents an adversary from decrypting or modify messages without our knowledge.

1 AI, External Resources, and Collaboration

AI Usage. You are permitted the use of AI to assist in syntax and debugging your program assignments. However, you are *not* allowed to have AI code up the entire assignment for you. You are also not copy code from libraries, implementations, or elsewhere. You should, using your knowledge from class and course materials, come up with the design of your program without the assistance of AI or other external resources. Violation of this policy is considered an honor code violation and may result in a failing grade for the assignment (and possibly the entire course).

Collaborations. Collaborating with classmates is encouraged and allowed. Code walks are individual, and each student will be expected to fully understand and be prepared to talk about their submissions on their own. Collaborations should be acknowledged in the design document submitted alongside the code.

2 Assignment Specification

In this assignment, you will be creating a secure messaging platform based on the secure messaging app Signal. You will have a server that users register public keys with. To send messages between users, users will communicate with the server to retrieve each others' public keys.

Server Authentication. For the base version of your assignment, **you do not need authentication between the users and the server.** That is, the adversary will not intercept or modify messages between the server. For extra credit, you may implement authentication. See extra credit section for more details.

Secure Messaging. For messaging between the users, there are three main functionalities your assignment should implement:

- Diffie-Hellman Key Exchange
- Authenticated Encryption
- Diffie-Hellman Ratchet

First, the users will perform a Diffie-Hellman Key Exchange to compute a shared key from their public keys. They will use this as input into a HMAC-based Key Derivation Function (HKDF) to derive 2 keys for the authenticated encryption: one for the encryption and one for the MAC. To get the two keys, you should run the HKDF twice with two different salts. These salts should be distinct but may be hard-coded public values. For the encryption, you should use AES in CBC-mode. For the MAC, you should use HMAC. For the inner and outer pads in the HMAC construction, see Section 2 and Section 5 of the FIPS Publication for HMAC.

Once users have initiated their conversation, they should rotate their keys using a Diffie-Hellman ratchet. For details, see Section 2.3 of the Signal Double Ratchet Algorithm Specification. For simplicity, do you *not* need to implement the symmetric-key ratchet, and users will alternate sending messages.

2.1 Main Program API Specification

The following detail the commands your program should handle and expected behavior. Parameters will be input in hex, and all other values will be input as strings. During testing and grading, we will not input any non-standard or weird character codes.

setup_params [p] [g] [q]

We will run this command to setup the public parameters: modulus p , group generator g , and group order q . All values will be given in hex.

init_user [userID]

This should start up a client, who samples a secret key k and computes the corresponding public key g^k . Output onto standard out the public key g^k .

register_user [userID]

The will only be run after `init_user` is run at least once for the specific `userID`. The user will send to the server its `userID` and public key, which the server will store. Output onto standard out all of the pairs of IDs and public keys registered to the server in the form `(userID, public_key)`. Each pair should be printed on a new line.

If a public key is already registered for `userID`, the server should overwrite the previous entry with the new public key.

get_user [userID]

Output the public key stored by the server for **userID**. If there is not entry for the ID, output the string “**userID is not registered.**”.

init_dm [senderID] [receiverID] [msg]

senderID should retrieve the public key of **receiverID** from the server, and the server will send the public key of **senderID** to **receiverID**. Users will use these public keys to perform a Diffie-Hellman Key Exchange. This shared key will be input into a HMAC-based Key Derivation Function (HKDF) to derive keys for the authenticated encryption.

Using authenticated encryption, **senderID** will encrypt and send **msg** to **receiverID**. **receiverID** should attempt to decrypt the message. If decryption fails, **receiverID** should output the string “**Message from senderID failed.**”. Otherwise output onto standard out the decrypted message.

send_dm [senderID] [receiverID] [msg]

This will only be run after **init_dm** has already been run between the two users. For simplicity, the roles of who is sender and who is receiver will alternate between messages.

The users should perform a Diffie-Hellman ratchet to derive new keys. Using these new keys, the sender will use authenticated encryption to send encrypt and send the message **msg** to the receiver. If decryption fails, **receiverID** should output the string “**Message from senderID failed.**”. Otherwise output onto standard out the decrypted message.

2.2 External Libraries

Allowed. For this assignment, you may use existing implementations of AES (including AES in CBC-mode), SHA-256, and HKDF. You are also allowed to use libraries for basic group operations (e.g., exponentiation, modular arithmetic, computing inverses) and sampling randomness.

Not Allowed. In this project you are expected to implement DH Key Exchange¹, HMAC (using SHA-256 as a building block), and DH Ratchet. While you may use an existing implementation of the encryption scheme, the combination of the encryption scheme and HMAC to form Authenticated Encryption should be done in your code.

Suggested Libraries. You are not required to use the following libraries, but they contain building blocks that may be helpful for your assignment.

- C++: CryptoPP or AWS libcrypto
- Rust: Any of the libraries withing Rust Crypto

2.3 Submission

Your assignment should be submitted through Gradescope as a ZIP. You will submit the following four components:

1. Source code for your implementation (written in either Rust or C++).
2. A Makefile that compiles your implementation upon running **make**.
3. A main program and file titled “**COMMAND**” that contains on one line the command to run your main program.

¹You may optionally use your implementation from HW 0.

4. A PDF titled “design” that includes your name, UNI, instructions for how to run your implementation by hand, and information about your implementation. You should include major design decisions, an outline of your code, and any other information you think is relevant to the grading of your assignment. If applicable, please also acknowledge any collaborators you worked with and if/how AI was used in your assignment.

Testing and Grading. For testing and grading, you will also submit a “main” program that can be used to interface with your implementation. Your “main” program may be in any language, but the your implementation for this assignment must be in Rust or C++. (It will likely be easiest to have your main program written in the same language as your implementation.)

The main program will read a sequence of newline delineated commands from standard input ending with EOF. When instructed to, it will display output to standard out. We will only use the main program in the way specified in the API specification and with valid inputs. We are not looking to trip you up on API edge cases.

Codewalks. There will be random codewalks throughout the semester where you will be expected to explain not only your code but also design decisions. Codewalks are only announced after submissions close.

3 Extra Credit

For extra credit, modify your assignment to prevent an adversary from posing as the server. You may hard-code a key pair for the server and assume each user has access to the server’s public key (i.e., you can directly load the server’s public key onto the user). In your design document, briefly describe the attack(s) your design change(s) address. Modifications do not need to handle all possible attacks.